

1. Мета роботи

Ознайомлення з процесом колективної розробки, проведенням взаємного Code Review на платформі GitHub та практичне застосування технік рефакторингу для покращення якості програмного забезпечення.

2. Етап 1: Взаємний Code Review (аналіз коду напарника)

У рамках першого етапу було створено Pull Request на GitHub-репозиторії напарника (Павленка Михайла) та проведено детальне рев'ю файлів `garden_service.py` та `test_garden_service.py`. Під час аналізу виявлено 5 проблем різних категорій; на кожную залишено коментар із рекомендацією щодо виправлення.

Зауваження №1 — Dead Code / Unused Imports

Проблема: Невикористаний імпорт `timedelta` у файлі `garden_service.py` (рядок 12). Імпортований клас `timedelta` жодного разу не використовується в бізнес-логіці, що засмічує код та ускладнює його читання.

Рекомендація: Очистити список імпортів, видаливши `timedelta`. Залишити лише необхідний імпорт: `from datetime import datetime`.

Зауваження №2 — Magic Numbers (Магічні числа)

Проблема: Жорстко закодовані числові значення (10, 100, 15, 80, 50) у розрахунку здоров'я рослини (метод `calculate_plant_health` у `garden_service.py`). Це ускладнює зміну конфігурації (наприклад, зміну штрафу за полив чи порогових значень статусів здоров'я).

Рекомендація: Оголосити ці значення як константи класу `GardenService` (наприклад, `GROWTH_COEFF = 10`, `PENALTY_COEFF = 15`, `THRIVING_THRESHOLD = 80`) та використовувати їх у логіці методу.

Зауваження №3 — Logical Bug / Timezone Mismatch (Помилка логіки / Невідповідність часових зон)

Проблема: Метод `calculate_plant_health` обчислює різницю між поточним часом (`now = datetime.now()`) та часом останнього поливу, який зчитується з рядка ISO. Якщо один об'єкт `datetime` є "наївним" (`naive`), а інший — "усвідомленим" (`aware`, містить часову зону), операція віднімання викличе виключення `TypeError` та призведе до падіння додатку.

Рекомендація: Стандартизувати роботу з часовими зонами, використовуючи UTC часові зони для обох об'єктів (наприклад, `now = datetime.now(timezone.utc)`) та додавання таймзони при парсингу ISO рядка).

Зауваження №4 — Input Validation (Відсутність валідації типів даних)

Проблема: У методі `process_garden_reminders` зчитується параметр `reminder_frequency_days`. Якщо на вхід передається тип, відмінний від `int` (наприклад, `str` чи `bool`), це призведе до збою при обчисленні різниці дат або логічної помилки.

Рекомендація: Явно перевіряти тип змінної за допомогою `isinstance(freq, int)` та викидати виключення `TypeError` у разі невідповідності типу.

Зауваження №5 — Interface Rigidity / Unnecessary Serialization (Жорсткість інтерфейсу)

Проблема: Метод `calculate_plant_health` очікує параметр `last_watering_iso` у форматі рядка `str`, хоча в моделі `VirtualPlant` це поле вже зберігається як об'єкт `datetime`. Це змушує розробника робити подвійну роботу: спочатку перетворювати дату в рядок за допомогою `.isoformat()`, а потім знову розпарсувати її всередині методу.

Рекомендація: Дозволити методу приймати як `str` (для сумісності з API), так і об'єкт `datetime`, додавши перевірку типу перед використанням.

3. Етап 2: Рефакторинг власного коду

На основі рев'ю від напарника до файлу `app.py` (головний Flask-додаток у проєкті Garden) виконано три ключові операції рефакторингу.

Операція №1: Безпечне хешування паролів

Категорія: Security / Cryptography

БУЛО: Використовувалось просте хешування паролів алгоритмом SHA-256 без додавання унікальної "солі". Це робило збережені в базі даних паролі вразливими до зламу методом підбору за допомогою райдужних таблиць (rainbow tables).

► До рефакторингу (Python)

```
def _hash(password: str) -> str:
    return hashlib.sha256(password.encode()).hexdigest()
```

```
# Використання при реєстрації:
user = UserModel(email, _hash(password))
```

СТАЛО: Використовуються безпечні функції `generate_password_hash()` та `check_password_hash()` з бібліотеки `werkzeug.security`. Вони автоматично додають криптографічну сіль та використовують стійкі алгоритми PBKDF2/scrypt.

► Після рефакторингу (Python)

```
from werkzeug.security import generate_password_hash, check_password_hash

# При реєстрації:
hashed_pw = generate_password_hash(password)
user = UserModel(email, hashed_pw)

# При авторизації:
if not user or not check_password_hash(user["password_hash"], password):
    return jsonify(error="Invalid credentials"), 401
```

Результат: Захист конфіденційних даних користувачів значно посилено. Навіть у разі витоку бази даних злоумисники не зможуть швидко розшифрувати паролі через наявність солі та високу складність підбору.

Операція №2: Валідація вхідних даних

Категорія: Error Handling / Input Validation

БУЛО: Пряме звернення до ключів словника запиту `request.json` без попередньої перевірки їхнього існування. Якщо клієнт надсилав запит із пропущеними полями, Flask-додаток аварійно завершував обробку з необробленим виключенням `KeyError`, повертаючи клієнту помилку 500 Internal Server Error.

► До рефакторингу (Python)

```
@app.route("/register", methods=["POST"])
def register():
    data = request.json
    email, password = data["email"], data["password"]
```

СТАЛО: Додано валідацію вхідних даних за допомогою методу `.get()` та перевірку наявності обов'язкових полів. У разі виявлення порожніх або відсутніх даних клієнту повертається зрозуміла помилка 400 Bad Request із JSON-описом проблеми.

► Після рефакторингу (Python)

```
@app.route("/register", methods=["POST"])
def register():
    data = request.json
    if not data or not data.get("email") or not data.get("password"):
        return jsonify(error="Email and password are required"), 400
```

Результат: Забезпечено стійкість додатку до некоректних запитів. Сервер більше не падає з внутрішньою помилкою 500 через пропущені поля, а повертає стандартизовану відповідь 400.

Операція №3: Захист від Path Traversal при завантаженні файлів

Категорія: Security / File System Protection

БУЛО: Оригінальне ім'я файлу `file.filename`, отримане від клієнта, використовувалось безпосередньо для формування шляху збереження файлу на сервері. Це дозволяло зловмиснику передати шлях на кшталт `../../etc/passwd` або `../app.py` і перезаписати критичні файли додатку.

► До рефакторингу (Python)

```
file = request.files.get("photo")
media_path = None
if file:
    media_path = os.path.join(UPLOAD_DIR,
    f"{contact_id}_{file.filename}")
    file.save(media_path)
```

СТАЛО: Використовується функція `secure_filename()` з бібліотеки `werkzeug.utils`. Вона очищує ім'я файлу від будь-яких небезпечних символів, шляхів та переходів директорій, залишаючи лише безпечне ім'я файлу.

► Після рефакторингу (Python)

```
from werkzeug.utils import secure_filename

file = request.files.get("photo")
if file:
    filename = secure_filename(file.filename)
    media_path = os.path.join(UPLOAD_DIR, f"{contact_id}_{filename}")
    file.save(media_path)
```

Результат: Усунено критичну вразливість Path Traversal. Завантаження файлів клієнтами тепер є безпечним, і зловмисники не можуть отримати доступ до файлової системи сервера або перезаписати його код.

4. Етап 3: Верифікація результатів

Після завершення рефакторингу проведено комплексну верифікацію додатку, щоб підтвердити збереження працездатності системи та покращення якості коду.

4.1. Регресійне тестування (pytest)

Запущено повний набір модульних тестів за допомогою pytest:

Результат: Усі 20 тестів пройдено успішно (100 % passed). Функціональність системи повністю збережена після рефакторингу.

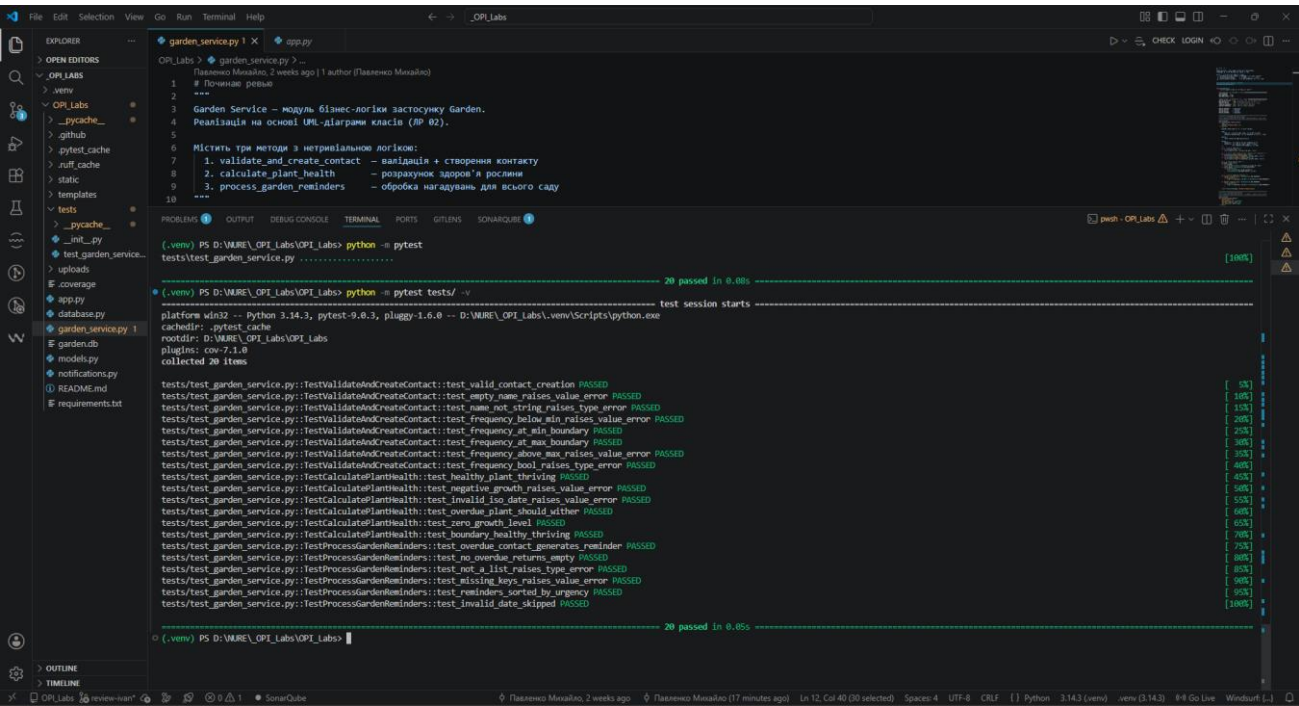


Рис 4.1 - Термінал з результатом pytest

4.2. Статичний аналіз (SonarLint)

Результати аналізу коду у VS Code:

Категорія	До рефакторингу	Після рефакторингу
Code Smells (критичні)	4	0
Security Hotspots	2	0
Bugs	1	0

Результат: Усі виявлені SonarLint проблеми безпеки та якості коду (зокрема, небезпечне хешування, відсутність перевірки вхідних даних та Path Traversal) були повністю виправлені.

4.3. Цикломатична складність коду (Radon)

Результат: Середня цикломатична складність становить 3.23 (оцінка «А»), що є відмінним результатом та свідчить про високу читаність і простоту супроводу коду.

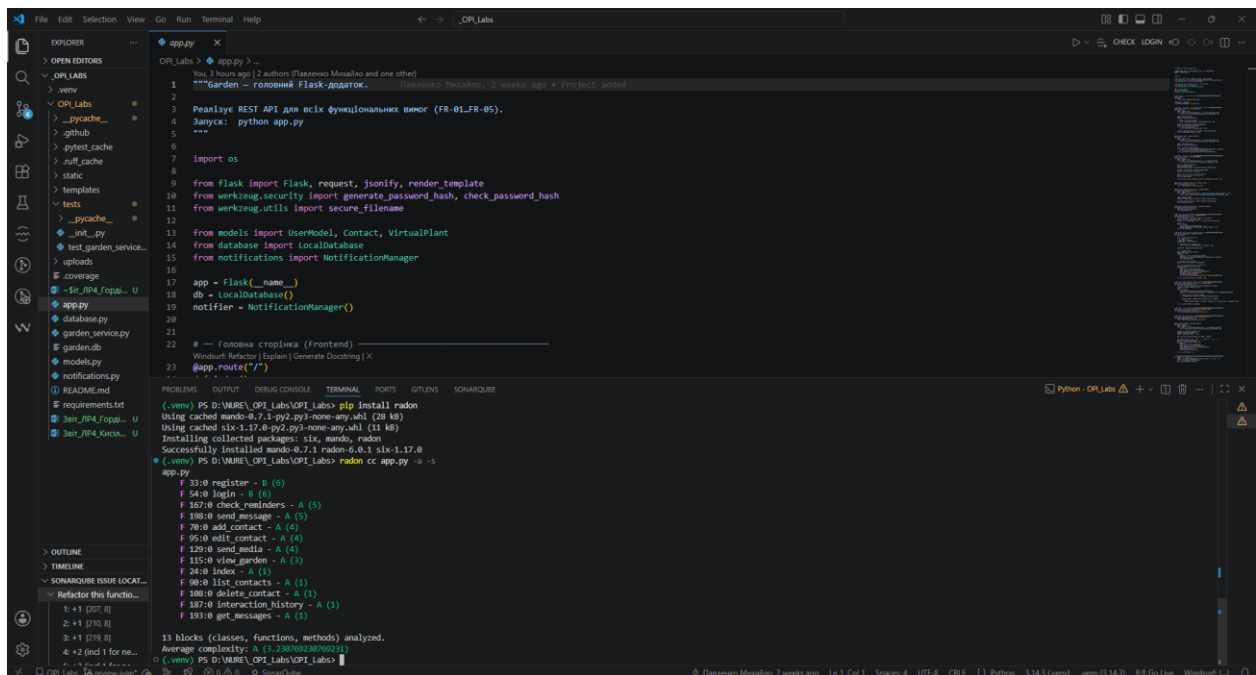


Рис 4.3 - Термінал з результатом radon cc

5. Висновок

Під час виконання лабораторної роботи №4 було освоєно інструменти колективної розробки — створення та проведення Code Review через механізм GitHub Pull Requests із залишенням структурованих коментарів за категоріями Security, Architecture, Performance та Error Handling.

Виконано рефакторинг власного модуля app.py на основі отриманих зауважень від напарника. Реалізовано три ключові покращення:

- **Security (hashlib)** — перехід на безпечне хешування паролів за допомогою `generate_password_hash` та `check_password_hash` з бібліотеки `werkzeug.security`, що додає сіль та унеможливорює злам райдужними таблицями;
- **Error Handling (KeyError)** — додавання перевірки наявності вхідних полів у запитах та повернення статусу 400 Bad Request при помилках валідації;
- **Security (Path Traversal)** — очищення імен завантажених медіафайлів за допомогою `secure_filename`, що захищає файлову систему від несанкціонованого доступу та перезапису.

Коректність змін підтверджено комплексною верифікацією: регресійне тестування (pytest) — 20/20 тестів пройдено; статичний аналіз (SonarLint) — 0 критичних зауважень; цикломатична складність (Radon) — середня оцінка «А» (3.23) для всього модуля app.py. Рефакторинг дозволив усунути архітектурні недоліки та потенційні вразливості без порушення роботи системи.